TopoGuard: Adaptive Topology Orchestration for Risk-Sensitive Decision Systems under Hard Constraints and Multimodal Uncertainty

ANONYMOUS AUTHOR(S)

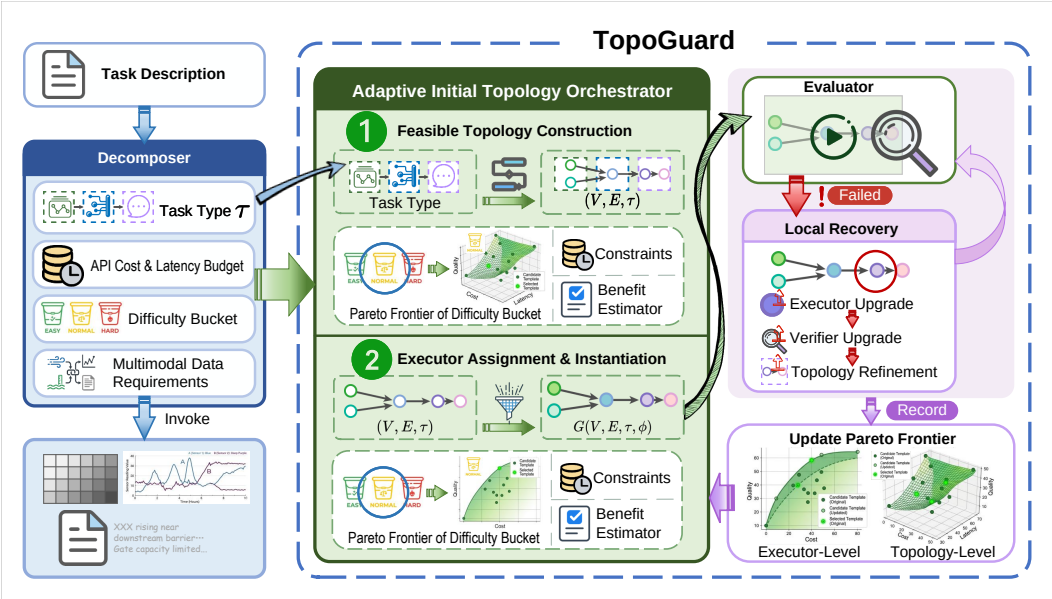

Fig. 1. Overview of TopoGuard. Given a task description, resource constraints, and heterogeneous multimodal evidence, TopoGuard selects an auditable workflow prototype, assigns executors, monitors intermediate outputs, and applies bounded local recovery when failures occur.

Risk-sensitive decision-support systems, such as storm-surge warning and emergency response, increasingly rely on multi-stage collaboration among heterogeneous modules and multimodal evidence, including textual requests, sensor time-series, and spatial forecast products. Such systems face a dual challenge: multimodal uncertainty, such as noisy sensors or conflicting weather patterns, can destabilize evidence reliability, while hard operational constraints, such as safety thresholds, strict time windows, and mandatory human oversight, make unconstrained trial-and-error or full online replanning difficult to justify. Fixed pipelines are easy to deploy but brittle across structurally diverse task instances. We propose **TopoGuard**, a profile-guided framework for workflow-structure-level adaptation in multimodal decision support. Instead of exhaustive online search, TopoGuard maintains task-conditional quality–cost–latency profiles for candidate workflow topologies (auditable DAG prototypes) and executor assignments, uses constrained Pareto selection to choose an initial topology aligned with the current task difficulty and operational constraints, and records execution feedback for future profile update. Evaluator modules can monitor intermediate output quality and trigger bounded local repair when failures occur; the repair procedure upgrades only affected stages rather than replanning the entire workflow. Using a trace-based evaluation with frozen profiles over held-out execution records from a water-conservancy digital-twin platform, TopoGuard nearly matches the quality-maximizing baseline ($S = 0.902$ vs. 0.914) while reducing latency (77.6 s vs. 118.9 s), and outperforms fixed pipelines, executor-only routing, and offline-search proxy baselines. These results suggest that auditable topology orchestration can serve as a practical structure-level adaptation mechanism for multimodal decision support,

achieving near quality-maximizing performance with lower latency and bounded recovery under operational constraints.

CCS Concepts: • **Information systems** → **Decision support systems**; **Multimedia information systems**; • **Computing methodologies** → *Artificial intelligence*; *Planning and scheduling*.

Additional Key Words and Phrases: Multimedia Systems, Topology Orchestration, Multimodal Decision Support, Workflow Adaptation, Local Graph Repair, Digital Twin

ACM Reference Format:
Anonymous Author(s). 2026. TopoGuard: Adaptive Topology Orchestration for Risk-Sensitive Decision Systems under Hard Constraints and Multimodal Uncertainty. 1, 1 (May 2026), 27 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern multimodal intelligent systems are increasingly used for decision support in risk-sensitive domains such as hydrological analysis, infrastructure monitoring, and emergency response [1, 13, 25, 31, 38, 40]. In these settings, task completion typically requires the coordinated use of heterogeneous functional modules, including data retrieval, numerical computation, reasoning, and verification [11, 34, 41]. System effectiveness therefore depends not only on individual executors, but also on how they are organized into an executable collaboration topology.

However, such systems face a dual challenge: multimodal uncertainty (e.g., noisy sensors or conflicting weather patterns) destabilizes evidence reliability, while hard constraints (e.g., safety thresholds, strict time windows, and mandatory human oversight) prohibit unconstrained online exploration. Many deployed systems still rely on manually designed workflows or fixed model-selection rules where functional nodes and execution order are predetermined [6, 19, 39, 42]. Such designs are easy to deploy but brittle in uncertain environments: different task instances require different processing structures; intermediate outputs may fail; and execution must satisfy constraints on quality, cost, and latency. A workflow that performs well for one instance may become expensive, slow, or ineffective for another, especially in risk-sensitive applications where local failures can propagate into unreliable downstream decisions [29]. This issue is further amplified in multimodal settings with missing or unreliable modalities [17, 20, 26, 33, 40, 43]. In these settings, trial-and-error exploration is often infeasible because failed executions may consume scarce response time or violate operational safeguards.

Existing approaches attempt to improve adaptability through tool use, reasoning-action loops, and parallel function scheduling [14, 27, 36]. However, the core decision is not only which executor to use, but also which functional nodes should be activated, how they should be connected, and how the execution graph should be adjusted when failures occur. In other words, the decision variable is the workflow topology itself. Recent work on LLM planning, graph execution, and workflow generation highlights structured execution organization, but mainly focuses on planning quality rather than topology adaptation under execution-time constraints [6, 10, 11, 19, 24, 34, 42]. We therefore formulate the problem as *adaptive topology orchestration* under hard constraints and multimodal uncertainty, focusing on how multimedia evidence streams are routed, verified, and selectively invoked rather than on learning a new cross-modal representation.

To address this problem, we propose **TopoGuard**, an adaptive topology orchestration framework for risk-sensitive multimodal decision support. TopoGuard treats workflow execution as a closed-loop decision process over structured graphs. It maintains task-conditional quality–cost–latency profiles for candidate auditable workflow DAG prototypes and executor assignments. Based on these profiles, TopoGuard performs constrained Pareto-based selection to obtain an initial executable graph aligned with the current task difficulty and operational constraints. During execution,

evaluator modules monitor intermediate outputs and detect local failures or reliability risks. Instead of triggering global replanning, TopoGuard performs bounded local graph repair that upgrades only affected stages while preserving reusable upstream computation.

A central observation motivating this work is that multimodal decision-support systems can benefit from choosing and maintaining appropriate *collaboration topologies*, rather than relying only on component selection within a fixed pipeline. Topology-level adaptation allows the execution graph to adjust to task requirements, resource constraints, and execution-time uncertainty, which is relevant for balancing effectiveness, efficiency, and reliability in risk-sensitive environments.

We evaluate TopoGuard on representative tasks involving heterogeneous executors and structured workflow operators. Experimental results show that TopoGuard nearly matches the quality-maximizing baseline while reducing latency, outperforming fixed workflows, executor-only routing, and single-objective baselines. Additional analyses indicate that bounded local repair provides useful recovery when intermediate execution quality degrades, and that topology-level adaptation is especially relevant when task instances differ in structural requirements.

The main contributions of this paper are summarized as follows:

- **Problem Formulation.** We formulate adaptive workflow execution in risk-sensitive multimodal decision support as adaptive topology orchestration over auditable workflow prototypes under hard resource constraints and heterogeneous multimodal evidence.
- **TopoGuard Framework.** We propose **TopoGuard**, a feasibility-aware topology orchestration framework that integrates offline performance profiling, constrained Pareto-based initial topology selection, and evaluator-triggered bounded local repair.
- **Empirical Validation.** We evaluate TopoGuard using a trace-based protocol with frozen profiles over recorded digital-twin execution traces and show that it nearly matches the quality-maximizing baseline while reducing latency, outperforming fixed pipelines, executor-only routing, and offline-search proxy baselines. Component analyses further indicate that topology selection and executor adaptation are the main average-performance drivers, while local repair provides targeted recovery after detected intermediate failures.

2 Related Work

2.1 Multimedia Decision Support and Digital Twins in High-Stakes Domains

Multimedia systems are increasingly used beyond perception and retrieval, and have become part of decision-support pipelines in high-stakes settings such as emergency response, infrastructure monitoring, and hydrological analysis [23, 29, 37]. In water and infrastructure domains, recent digital-twin studies emphasize real-time monitoring, simulation, and decision support as core capabilities of intelligent management systems [8, 18]. These systems typically integrate heterogeneous observations, simulation outputs, and operational knowledge, but their execution logic is often implemented as manually designed pipelines or domain-specific control flows. Our work is related to this line of research in that it also targets decision support under multimodal inputs and operational constraints. The difference is that we focus on *how* the execution structure should be selected and adjusted during execution when task requirements, resource limits, and intermediate reliability vary across instances.

2.2 Multimodal Uncertainty and Missing-Modality Robustness

A large body of multimodal research studies robustness under missing, noisy, or unreliable inputs [17, 33]. This literature is directly relevant because real-world decision systems often operate with partial observability, degraded sensors, or cross-source inconsistency. Existing methods mainly improve robustness within a fixed model architecture or a fixed processing pipeline, for example

through missing-modality learning, representation completion, or robust fusion. By contrast, we examine a different question: when evidence quality changes, should the system still use the same execution structure, or should it switch to a different workflow topology with additional verification or recovery stages? In this sense, our work complements model-level robustness methods by addressing structure-level adaptation.

2.3 Workflow and Agent Orchestration

Recent work on LLM-based agents has highlighted the value of structured execution rather than single-shot prompting. ReAct and Tree of Thoughts illustrate how intermediate reasoning and tool interaction can improve complex task solving [35, 36]. More recent workflow-oriented approaches explicitly treat agent execution as a graph or workflow construction problem. For example, AFlow formulates agentic workflow generation as search over code-represented workflows and optimizes node-edge structure automatically [42]. These studies show that execution structure matters, but they primarily focus on producing strong workflows or better reasoning performance. Our setting is different in two respects: we consider hard deployment constraints on cost and latency during selection, and we explicitly model bounded local repair after execution has started.

2.4 Cost-Aware Routing, Cascading, and Runtime Recovery

Another closely related line of research studies cost-aware model selection and cascading. FrugalGPT shows that routing or cascading across models can reduce inference cost while maintaining strong task performance [3]. Subsequent work further unifies routing and cascading and highlights the importance of quality estimation for cost-performance trade-offs [5, 7, 21]. These methods are relevant because they also optimize quality under budget constraints, but they usually treat the decision variable as *which model* to call for a given query. Our work instead treats the workflow topology itself as the decision variable, jointly considering node activation, graph structure, executor assignment, and local repair.

Our repair component is also related to the broader literature on workflow exception handling and adaptive failure recovery. Prior research in workflow systems and service composition has studied exception handling, ad-hoc recovery, online recovery, and adaptive failure handling for composite processes [4, 12, 15, 32]. Those studies motivate the importance of execution-time recovery, but they are generally not formulated around modern multimodal agent workflows or multi-objective quality-cost-latency trade-offs. We build on the same general intuition that failures should be handled locally when possible, while instantiating it in a profile-guided orchestration framework for multimodal decision systems.

3 Method

3.1 Problem Setup

We consider adaptive orchestration for multi-stage decision tasks. For an input task X, the system must choose (i) a workflow structure, (ii) executor assignments for the nodes in that structure, and (iii) a limited set of repair actions if execution quality falls below a threshold.

The framework has two coupled stages: initial topology selection under hard cost–latency constraints and quality-oriented utility, followed by bounded local repair when intermediate failures are detected. Because the workflow-graph space is combinatorial, direct search is impractical; we therefore use a hierarchical selection procedure together with minimal-change local repair.

Multimodal Input and Output Specification. The system operates on heterogeneous multimodal inputs and produces multi-format outputs. Concretely, the input to a task instance comprises:

- **Sensor time-series data**: hydrological measurements (water level, flow rate, temperature) sampled at regular intervals.
- **Forecast products**: heterogeneous meteorological outputs including spatial fields (wind velocity, precipitation maps), scalar indices (storm surge height, wave period), and probabilistic forecast envelopes.
- **Textual queries**: natural-language task descriptions, user questions, or domain-specific instructions.

These multimodal inputs are passed through workflow nodes that may consume different modality combinations. For example, a retrieval node processes textual queries against a knowledge base; a computation node takes sensor time-series and produces scalar or vector diagnostics; a reasoning node consumes both textual context and numerical outputs from upstream computation. The workflow output similarly spans multiple modalities: temporal evolution curves, spatial comparison maps, statistical summaries, and natural-language warnings or explanations. This heterogeneous input–output structure is characteristic of the multimedia decision-support setting and motivates the need for topology-level adaptation: different modality combinations may call for different processing structures depending on which modalities are available, noisy, or conflicting during execution.

3.2 Initial Topology Selection under Hard Constraints

We first formulate the construction of the initial executable workflow as a constrained topology selection problem, following the general multi-objective view that operational decisions should be evaluated jointly over utility, resource use, and feasibility [9, 22, 28, 30]. Given an input task X, the policy π selects an initial workflow graph $G_\pi(X)$ from the candidate graph space. The objective is to maximize utility under hard structural and resource constraints:

$$\pi^\star = \arg\max_{\pi \in \Pi} \mathbb{E}_{X \sim D}\big[Q(G_\pi(X), X)\big], \tag{1}$$

subject to

$$\begin{aligned} &G_\pi(X) \in \mathcal{G}_{\mathrm{DAG}}, \\ &C(G_\pi(X), X) \le B(X), \\ &L(G_\pi(X), X) \le \Delta(X). \end{aligned} \tag{2}$$

Here, $\mathcal{G}_{\mathrm{DAG}}$ denotes the space of valid directed acyclic workflow graphs, $B(X)$ is the task-specific budget constraint, and $\Delta(X)$ is the latency constraint. This stage determines only the *initial* executable topology; repair actions during execution are formulated separately in Section 3.6.

To compare quality, cost, and latency on a comparable scale, we use a normalized utility:

$$Q(G, X) = \alpha\, S_{\mathrm{norm}}(G, X) - \beta\, C_{\mathrm{norm}}(G, X) - \gamma\, L_{\mathrm{norm}}(G, X), \tag{3}$$

where $\alpha, \beta, \gamma \ge 0$ and $\alpha + \beta + \gamma = 1$. In our implementation, quality is normalized as

$$S_{\mathrm{norm}}(G, X) = \frac{S(G, X)}{S_{\mathrm{scale}}}, \tag{4}$$

and cost and latency use the normalized profile values C_{norm} and L_{norm} for candidate comparison. Unless otherwise specified, we use $\alpha = 0.65$, $\beta = 0.25$, $\gamma = 0.10$, and $S_{\mathrm{scale}} = 1.5$. Pareto frontiers are used only for candidate pruning before the final utility-based selection.

3.3 Workflow Representation

A workflow is represented as a directed graph

$$G = (V, E, \tau, \rho, \phi), \tag{5}$$

where V is the node set, $E \subseteq V \times V$ is the directed edge set, $\tau(v)$ is node role type, $\rho(v)$ is executor function type, and $\phi(v)$ is runtime configuration. DAG topology is enforced by a topological order $\mathrm{ord} : V \to \{1, \ldots, |V|\}$ such that

$$(u, v) \in E \Rightarrow \mathrm{ord}(u) < \mathrm{ord}(v). \tag{6}$$

Task context is encoded into a discrete difficulty level $b \in \{\text{easy}, \text{medium}, \text{hard}\}$, estimated from task features such as query length, domain complexity, and retrieval scope. The resulting difficulty level is used to condition both template retrieval and executor candidate selection.

In the current implementation, topology adaptation is instantiated as selection over a controlled space of auditable workflow DAG prototypes rather than unrestricted DAG generation. Each prototype is defined over typed functional nodes, including retrieval, reasoning, computation, verification, and aggregation, and corresponds to a valid executable topology with profiled quality–cost–latency behavior. The four prototypes used in our experiments (direct, fallback-direct, executor–verifier, and executor–verifier–aggregator) are representative structures in this controlled topology space. This design is intentional for risk-sensitive deployments, where candidate structures must be inspectable, profileable, and safe to execute. Our claim is therefore template-guided adaptive topology orchestration over auditable DAG prototypes, not arbitrary topology learning from scratch; larger template libraries, including parallel or hierarchical structures, can be incorporated once sufficient profile data are available. The framework itself is agnostic to the size of the template library: the same selection procedure applies whether the candidate pool contains 4 prototypes or 40.

3.4 Profile Learning

TopoGuard uses historical execution traces to estimate two coupled kinds of profiles. A *topology-prototype profile* summarizes the expected quality, cost, and latency of a candidate DAG prototype under a task context, while an *executor profile* summarizes the same quantities for assigning a concrete executor to a typed workflow node. Thus, the profile is not attached only to an executor or only to a topology node; it is indexed by the tuple

$$(\text{node type}, \text{difficulty}, \text{topology prototype}, \text{executor}).$$

For each tuple observed in the training split, we aggregate execution records to obtain

$$\widehat{S} = \mathrm{mean}(S), \qquad \widehat{C} = \mathrm{mean}(C), \qquad \widehat{L} = \mathrm{mean}(L),$$

together with empirical standard deviations and sample counts. Cost and latency are additionally log-normalized for Pareto filtering and utility ranking, while raw values are retained for reporting. Topology-level selection marginalizes over the profiled candidates belonging to the same prototype and context; node-level selection then chooses the executor assignment within the selected prototype. During evaluation, these profiles are frozen: all strategies choose from profile estimates, and held-out realized quality, cost, and latency are used only after a candidate has been selected. Execution feedback is logged as future profile data but is not used to update the profile within the same test batch.

3.5 Difficulty-Aware Hierarchical Initialization

Given task X and difficulty level b, initialization is decomposed into two levels:

$$T_0 = \pi^T_{\text{init}}(X, b), \qquad \Phi_0 = \pi^N_{\text{init}}(X, b, T_0), \qquad G_0 = (T_0, \Phi_0). \tag{7}$$

At the topology-prototype level, we first perform Pareto pruning over candidate workflow DAG prototypes:

$$\mathcal{P}_T = \mathrm{Pareto}_{\max S, \min C, \min L}(\mathcal{T}(X, b)), \tag{8}$$

and then select the feasible topology prototype with highest proxy utility:

$$T_0 = \arg \max_{T \in \mathcal{P}_T^{\text{feasible}}} \widehat{Q}_T(T; X, b). \tag{9}$$

At the node level, each executor node retrieves a difficulty-conditioned candidate pool,

$$\mathcal{A}_v(b, \rho(v)), \qquad \mathcal{P}_N(v) = \text{Pareto}_{\max S, \min C}(\mathcal{A}_v(b, \rho(v))), \tag{10}$$

and selects the feasible candidate with maximal utility:

$$Q_N(a; v, X, b) = \alpha_N S(a; v, X, b) - \beta_N C(a; v, X, b), \tag{11}$$

$$\phi(v) = \arg \max_{a \in \mathcal{P}_N^{\text{feasible}}(v)} Q_N(a; v, X, b). \tag{12}$$

3.6 Bounded Local Repair with Edit-Loss

After the initial workflow G_0 has been selected and instantiated, execution proceeds in a closed loop. When the evaluator detects that an intermediate result fails the pass condition, TopoGuard does not re-optimize the whole graph from scratch. Instead, it performs bounded local repair around the affected region. This repair module is an integral execution-time adaptation layer of TopoGuard: initial topology orchestration chooses the starting workflow, while bounded repair maintains that workflow when local execution quality degrades.

At execution step t, let G_t denote the current workflow graph and s_t the local execution state. A repair action is selected as

$$a_t = \pi_{\text{repair}}(s_t), \qquad G_{t+1} = \mathcal{T}_{\text{op}}(G_t, a_t), \tag{13}$$

where $\mathcal{T}_{\text{op}}$ applies a local graph edit to the current workflow.

We consider four concrete bounded repair routes,

$$\mathcal{A}_t = \{A, B, C_1, C_2\}, \tag{14}$$

where A denotes topology/template upgrade, B denotes executor upgrade within the current topology, C_1 denotes cross-topology executor upgrade, and C_2 denotes verifier/evaluator upgrade. To discourage unnecessary large edits, we define the edit loss as

$$\mathcal{L}_{\text{edit}}(a) = \lambda_n \Delta n(a) + \lambda_\phi \Delta\phi(a) + \lambda_e \Delta e(a), \tag{15}$$

where $\Delta n(a)$, $\Delta\phi(a)$, and $\Delta e(a)$ measure changes in nodes, executor assignments, and edges. In our implementation, the weights are set to $\lambda_n = 0.30$, $\lambda_\phi = 0.25$, $\lambda_e = 0.10$, and the overall penalty weight is $\lambda = 0.20$.

The repair policy then solves a bounded local optimization problem:

$$a_t^\star = \arg \max_{a \in \mathcal{A}_t^{\text{feasible}}} \Big(Q(G_{t+1}(a), X) - \lambda\, \mathcal{L}_{\text{edit}}(a) \Big), \tag{16}$$

where the feasible repair set is

$$\mathcal{A}_t^{\text{feasible}} = \{a \in \mathcal{A}_t \mid S(G_{t+1}(a), X) \geq \tau_{\text{pass}}\}. \tag{17}$$

Here $\tau_{\text{pass}} = 0.5432$ is the data-driven repair trigger threshold (the 25th percentile of quality scores across training-profile entries in the Water QA benchmark), which is computed once from training data and held fixed at evaluation time. During repair selection, S and Q are profile-based estimates; held-out realized quality is used only after the repaired workflow has been selected.

This formulation separates repair from initial topology selection: the initial stage chooses a globally feasible workflow under hard constraints, whereas repair performs *local correction* aimed at restoring execution quality without global replanning.

Algorithm 1 TopoGuard Hierarchical Pareto Orchestration

Require: Task X, performance profile $\mathcal{P}$, budget B, latency bound Δ, repair threshold τ_{pass}
Ensure: Final workflow G^* and execution result

$b \leftarrow \textsc{EstimateDifficulty}(X)$ ▷ estimate task difficulty level
$\mathcal{F}_T \leftarrow \textsc{ParetoFilter}(\mathcal{P}, b, B, \Delta)$ ▷ topology-prototype-level Pareto pruning
$T_0 \leftarrow \arg\max_{T \in \mathcal{F}_T} \hat{Q}(T; X, b)$ ▷ select highest-utility topology prototype
for each $v \in V(T_0)$ **do**
  $\mathcal{F}_v \leftarrow \textsc{ParetoFilter}(\mathcal{P}, v, b, B)$ ▷ node-level Pareto pruning
  $\phi(v) \leftarrow \arg\max_{a \in \mathcal{F}_v} Q_N(a; v, X, b)$ ▷ select best executor
end for
Execute workflow $G_0 = (T_0, \Phi_0)$, obtain intermediate outputs $\{o_v\}$
for each $v \in V(T_0)$ **do**
  if $\textsc{Evaluator}(o_v) < \tau_{\text{pass}}$ **then** ▷ evaluator detects local failure
    $\mathcal{A}_{\text{feas}} \leftarrow \{a \in \{A, B, C_1, C_2\} \mid \widehat{S}(G(a), X) \geq \tau_{\text{pass}}\}$
    $a^\star \leftarrow \arg\max_{a \in \mathcal{A}_{\text{feas}}} \left[\widehat{Q}(G(a), X) - \lambda \mathcal{L}_{\text{edit}}(a)\right]$
    $G_0 \leftarrow \mathcal{T}_{\text{op}}(G_0, a^\star)$ ▷ apply bounded local repair
  end if
end for
Log execution feedback for future profile updates; **return** $G^* \leftarrow G_0$

4 Experiments

We evaluate TopoGuard using a trace-based protocol with frozen profiles over held-out execution records collected from a water-conservancy digital-twin platform. For each task instance, the system first selects an executable workflow under quality–cost–latency constraints and then monitors execution for possible local repair. The experiments are organized around the following questions:

- **RQ1:** Can TopoGuard achieve a strong quality–cost–latency trade-off relative to the baselines?
- **RQ2:** Do topology-prototype selection and executor adaptation provide complementary gains over fixed, routing-only, and offline-searched workflows?
- **RQ3:** Do the observed advantages remain stable across task domains and different utility preferences?
- **RQ4:** When intermediate execution quality degrades, can bounded local repair provide targeted recovery without global replanning?

4.1 Experimental Setup

Platform and task domains. We conduct all experiments on a water-conservancy digital-twin platform and evaluate TopoGuard on two representative task domains: **Water QA** and **Storm Surge**. The primary benchmark, **Water QA**, is formulated as a multi-stage question answering task over hydrological knowledge. Different task instances may require different execution structures, such as retrieval, reasoning, computation, verification, and aggregation. Multimodal uncertainty is reflected in both the evidence interface and the task difficulty dimension: easy, medium, and hard instances differ in query ambiguity, retrieval noise, and the reliability of intermediate outputs, while storm-surge tasks additionally involve heterogeneous forecast artifacts such as temporal curves, spatial maps, scalar metrics, and textual warning requirements. The benchmark is therefore designed around structure-level multimodal decision support: it evaluates whether the workflow

Table 1. Benchmark construction statistics for the Water QA domain.

Item	Count / Value
Candidate models	17
Node types	5 (retrieval, reasoning, computation, verification, aggregation)
Difficulty levels	3 (easy, medium, hard)
Topology templates	4 (direct, fallback-direct, executor+verifier, executor+verifier+aggregator)
Profile entries	163
Pareto frontier size	48
Training records	1,467
Test contexts (unique)	255 (221 feasible under default constraints)
Test episodes (total)	834
Constraint budget $C_{\max}$	0.5 (log-normalized)
Latency budget $L_{\max}$	0.90 (log-normalized)
Q-score scale S_{scale}	1.5

topology should adapt when different modality combinations and reliability levels are required. The auxiliary benchmark, **Storm Surge**, is used to examine whether the same orchestration rule can be applied to another decision domain without redesigning the core decision logic.

Data and evaluation protocol. For Water QA, we construct a benchmark containing training and test executions, which are further aggregated into historical performance profiles for candidate workflows and executors. For Storm Surge, we build an auxiliary transfer benchmark with heterogeneous execution choices and evaluate whether the same orchestration strategy can be applied to a second task domain without redesigning the decision rule. In both domains, the training split is used to estimate the expected quality, cost, and latency of candidate workflows, while all final results are reported on held-out execution records. Profiles are frozen during evaluation; execution feedback is logged for future updates but is not used to update profiles within the same test batch. The reported S, cost, and latency are therefore realized held-out outcomes after a candidate workflow has been selected, not the profile estimates used for selection. Thus, the main Water QA and Storm Surge results are trace-based recorded-execution evaluations with frozen profiles over platform executions, not purely synthetic simulations; only the stress and perturbation diagnostics inject controlled test-time degradations.

Given a test context, the system first queries historical profiles to estimate the expected triplet (S, C, L) for each candidate topology or executor, applies hard constraint filtering, computes the feasible Pareto frontier, and then selects an execution plan according to the corresponding orchestration strategy. For TopoGuard, the selected plan is further executed in a closed loop with evaluator monitoring and optional local repair. The repair trigger uses intermediate realized quality signals during execution; it does not observe the final held-out quality before deciding whether to repair.

Table 1 summarizes the benchmark construction statistics. The Water QA benchmark comprises 17 models $\times$ 5 node types $\times$ 3 difficulty levels, yielding 163 non-empty profile entries. Some rare combinations lack feasible candidates after hard cost–latency filtering; under the default constraints, 221 of 255 held-out contexts have at least one feasible candidate and are included in the main feasible-set comparison. The global Pareto frontier contains 48 non-dominated feasible profile candidates. All profiles are estimated from training records on the same platform; no external data is used.

The multimodal nature of the benchmark is reflected in both input and output artifacts: textual queries, hydrological time-series, spatial forecast fields, scalar measurements, and heterogeneous outputs including curves, maps, summaries, and warning text. Different workflow nodes consume different modality combinations, which is why topology selection rather than executor routing alone is central to the evaluation.

Compared methods. We compare seven orchestration methods, including TopoGuard and six baselines, organized by whether they select from the Pareto feasible candidate pool:

Candidate-pool selection methods (selecting from the profile-derived feasible candidate pool; the Water QA global Pareto frontier contains 48 non-dominated candidates):

- **TopoGuard (dual-layer adaptive):** performs Pareto pruning and utility-maximization selection at both the template level and the executor level, with bounded local repair. This is the full framework proposed in this paper.
- **LLM Router (executor-only adaptive):** selects the best executor by estimated quality within a fixed topology template (executor+verifier), without adapting workflow structure. Models the common model-routing approach and isolates the contribution of topology-level adaptation.
- **AFlow-Style (offline-searched):** searches for a globally optimal (topology, executor) combination on the training split using the same utility function, then fixes this workflow for all test contexts without task-conditional adaptation. This baseline is inspired by offline workflow-search methods such as AFlow and AutoFlow, but is instantiated within our controlled auditable topology space and does not constitute a full reproduction of [42], which targets unconstrained code-generation and planning workflows. It therefore serves as a proxy baseline for fixed offline workflow search rather than a direct reproduction.
- **Best-Quality / Profile-Cheapest:** select the candidate with the highest estimated quality or the lowest estimated cost, respectively. Because reported cost is realized execution cost, FrugalGPT Cascade can occasionally have lower realized cost than Profile-Cheapest.

Fixed-pipeline methods (not selecting from the candidate pool):

- **Static Workflow:** fixes the topology to executor+verifier and the executor to Kimi-K2.5. No Pareto screening, no adaptive topology selection, and no repair. This conservative non-adaptive pipeline serves as an alternative operating point for evaluating the quality-overhead trade-off.
- **FrugalGPT Cascade:** sorts executors by cost and selects the first meeting $S \geq 0.70$, without topology adaptation. Models the "call cheap model first, escalate only if needed" strategy.

This organization highlights the core innovation of TopoGuard—dual-layer adaptive topology selection—and provides controlled comparisons that isolate the contribution of each adaptive layer. The AFlow-Style comparison tests whether task-conditional adaptation provides value beyond offline workflow search, while the Static Workflow comparison evaluates whether the adaptive overhead is worthwhile in practice.

Metrics. We report three primary metrics: **quality** $S \in [0, 1]$, **cost** C (USD), and **latency** L (seconds). Quality measures the final task-solving effectiveness of the selected workflow. Cost is computed from actual execution expenses, and latency records the realized end-to-end execution delay. Following the current implementation, cost and latency are log-normalized only for internal Pareto filtering, while all reported results use raw values for interpretability.

The quality score S is computed with the same evaluator protocol for all compared methods. For each executed node or final workflow output, the evaluator applies the task-appropriate rubric and normalizes the resulting score to $[0, 1]$. Objective sub-tasks use direct task metrics when available;

open-ended decision-support outputs use judge-proxy rubrics covering correctness, completeness, constraint satisfaction, and clarity. Workflow-level S is the realized held-out quality attached to the selected execution record, whereas profile-level $\widehat{S}$ is estimated only from training records and used for selection. Thus, the reported S values are not observed by any strategy before choosing a candidate.

We do not report violation rate as a discriminative metric in the main comparison because feasibility filtering is applied before selection; however, feasible coverage is reported because some contexts have no candidate under the hard filters. Note on constraint scope: our experiments apply a single global cost budget $C_{\max} = 0.5$ and latency budget $L_{\max} = 0.9$ (log-normalized) uniformly across all 255 test contexts, rather than per-context constraints. This corresponds to a deployed platform setting where all tasks share the same operational resource ceiling. The constraint stress test (Section 4.6) varies $C_{\max}$ around the default value to evaluate robustness to moderate budget changes.

The internal selection metric $Q(G; X) = \alpha S_{\text{norm}} - \beta C_{\text{norm}} - \gamma L_{\text{norm}}$ is used solely for Pareto frontier ranking and candidate tie-breaking; it does not appear as a reported evaluation metric. $S_{\text{norm}} = S/S_{\text{scale}}$ normalizes quality to the same scale as C_{norm} and L_{norm}, preventing quality from dominating the score purely due to scale differences. In our implementation, $(\alpha, \beta, \gamma) = (0.65, 0.25, 0.10)$ with $S_{\text{scale}} = 1.5$, which balances quality prioritization with genuine cost-latency trade-offs on the Pareto frontier.

Experiment protocol. Our experiments consist of two complementary evaluation protocols: **(1) Shared-pool controlled comparison**: Best-Quality and Profile-Cheapest are evaluated as controlled selection baselines on the same learned profiles, selecting from the shared feasible candidate pool. This isolates the effect of the decision heuristic while holding the candidate space fixed. **(2) Operating-point comparison**: TopoGuard and Static Workflow are evaluated on the same test contexts but represent different deployment philosophies (adaptive vs. fixed). They are not compared on a single metric; instead, the two are contrasted on their respective strengths: TopoGuard on quality and adaptive robustness, Static Workflow on cost-efficiency and simplicity.

Implementation details. The full TopoGuard pipeline consists of: (1) initial context analysis, (2) workflow-level candidate screening, (3) node-level executor selection, (4) execution and evaluator monitoring, and (5) bounded local repair. The repair operators are defined as four bounded routes: topology/template upgrade (operator A), executor upgrade within the current topology (operator B), cross-topology executor upgrade (operator C1), and verifier/evaluator upgrade (operator C2). In the training-profile recorded-execution replay diagnostic, 231 of 585 node-level execution events trigger repair monitoring (39.5%); this diagnostic is reported separately from the main held-out strategy comparison in Section 4.5.

4.2 Main Results on Water QA

We first examine **RQ1**, namely whether TopoGuard achieves a strong quality–cost–latency trade-off on the Water QA benchmark.

Table 3 reports the trace-based evaluation results with frozen profiles on the feasible held-out Water QA contexts. The benchmark contains 255 held-out contexts and 834 held-out execution records; 221 contexts have at least one candidate after hard cost–latency feasibility filtering and are included in the main comparison.

Figure 2 visualizes the same trend as Table 3: TopoGuard stays close to Best-Quality in quality while avoiding the highest-cost operating point.

TopoGuard (adaptive) reaches $S = 0.902$ through profile-guided topology and executor selection. Best-Quality achieves a marginally higher quality ($S = 0.914$), but with higher latency (118.9 s

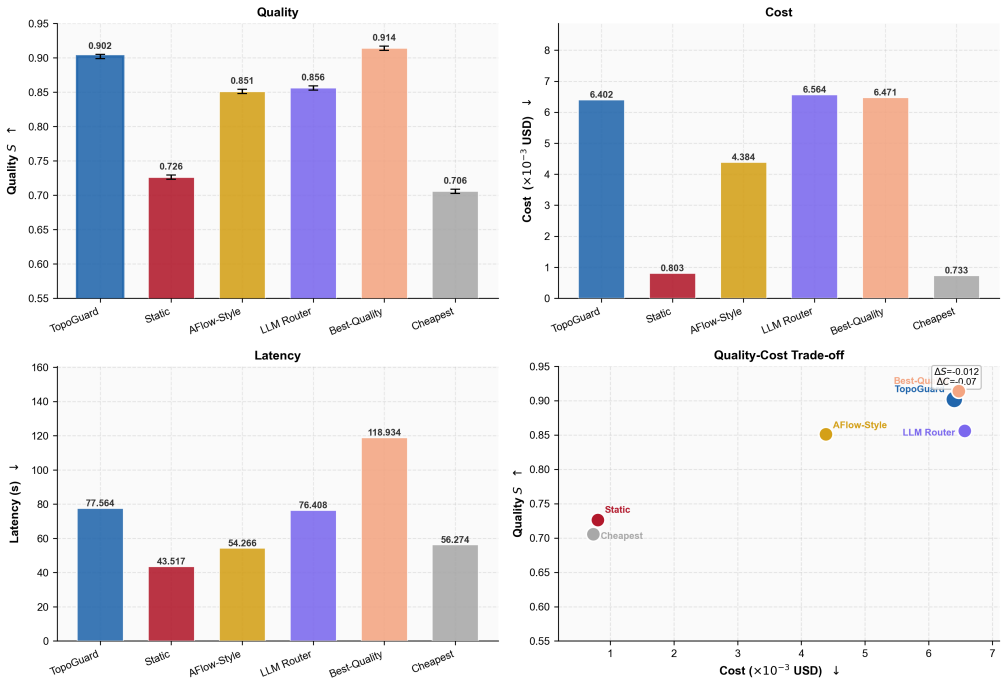

Fig. 2. Overall Water QA comparison. TopoGuard nearly matches the quality-maximizing profile-based baseline while improving over fixed, cascade, routing-only, and offline-search proxy baselines.

vs. 77.6 s) and similar cost (6.47×10^{-3} vs. 6.40×10^{-3} USD). Thus, TopoGuard should be interpreted as a balanced operating point rather than a single-metric optimum. On paired per-context comparison against Static Workflow ($n = 221$, 204 wins / 0 ties / 17 losses), TopoGuard leads with a mean advantage of $\Delta S = +0.176$ (0.902 vs. 0.726).

Cost-adjusted paired regression. To examine whether the quality gain is merely caused by higher resource use, we fit a paired regression between quality improvement and cost increase across the 221 feasible paired contexts: $\beta_0 = +0.1266$ (intercept), $\beta_1 = +8.8038$ (marginal return), with Cohen's $d = 1.0704$ (large effect). The positive intercept suggests that TopoGuard's advantage cannot be explained solely by additional cost; the quality improvement includes a component unrelated to resource investment. We treat this analysis as supportive rather than causal evidence, and use the component ablation in Section 4.5 to more directly isolate the roles of executor adaptation and topology-prototype selection.

Static Workflow (fixed) represents a conservative baseline with the executor+verifier topology and Kimi-K2.5 executor, without adaptive selection. The choice of executor+verifier (2-node) rather than executor+verifier+aggregator (3-node) reflects typical Water QA task requirements, where aggregation is not always necessary. It achieves lower quality ($S = 0.726$) but also lower cost ($C = 8.03 \times 10^{-4}$ USD) and latency ($L = 43.5$ s). The comparison therefore should not be read as TopoGuard dominating every operating point; rather, TopoGuard moves the feasible operating point toward substantially higher quality under the same profile-filtered candidate library.

FrugalGPT Cascade and **LLM Router** represent established baselines from the literature. FrugalGPT Cascade ($S = 0.757$, $C = 0.953 \times 10^{-3}$ USD, $L = 67.9$ s) escalates greedily through executors ordered by cost until a quality threshold is met, without any topology adaptation. LLM

Table 2. Equal-budget fixed-topology comparison. Win rate and ΔS are computed over contexts where the fixed topology has a candidate under TopoGuard's per-context selection cost ceiling.

Fixed topology	TopoGuard win rate	ΔS	Cost ratio
direct	93.4%	+0.110	matched
executor+verifier	65.0%	+0.044	matched
executor+verifier+aggregator	0.0%	−0.030	matched

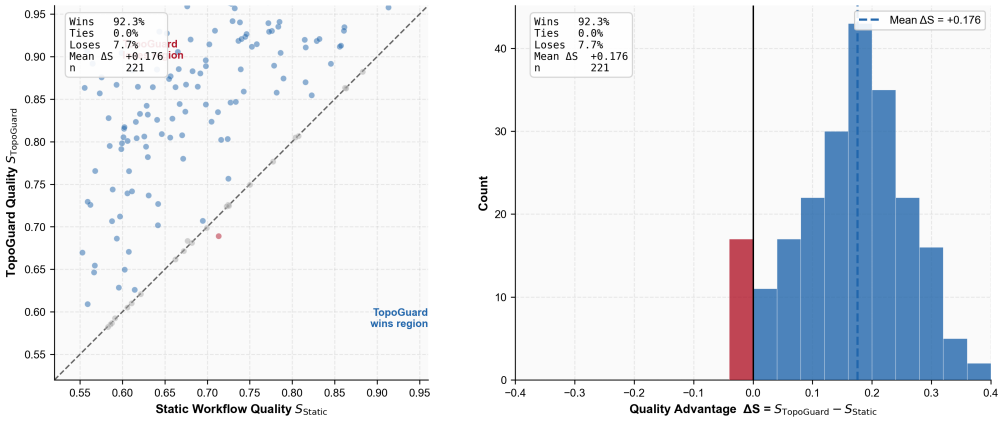

Fig. 3. Per-context comparison of TopoGuard and Static Workflow. TopoGuard wins in 204/221 paired feasible contexts (92.3%) with a mean quality advantage of $\Delta S = +0.176$.

Table 3. Closed-loop topology orchestration results on Water QA. All methods are evaluated on the same feasible held-out contexts after cost–latency filtering. Candidate-pool methods use the same profile-derived feasible pool. Best-Quality selects the candidate with the highest estimated quality S from this pool and represents the quality-maximizing operating point achievable through profile-based selection without a utility trade-off. AFlow-Style is a fixed offline-search proxy, not a full reproduction of [42].

Method	Quality $S \uparrow$	Cost ($\times 10^{-3}$ USD) $\downarrow$	Latency (s) $\downarrow$	N
TopoGuard	0.902	6.40	77.6	221
AFlow-Style Offline Search [42]	0.851	4.38	54.3	221
LLM Router [14]	0.856	6.56	76.4	221
Static Workflow	0.726	0.80	**43.5**	221
FrugalGPT Cascade [3]	0.757	0.95	67.9	221
Best-Quality	**0.914**	6.47	118.9	221
Profile-Cheapest	0.706	**0.73**	56.3	194

Router ($S = 0.856$, $C = 6.56 \times 10^{-3}$ USD, $L = 76.4$ s) selects the best executor within a fixed topology template, adapting only the model choice. The gap between LLM Router and TopoGuard ($\Delta S \approx +0.046$) isolates the contribution of topology-level adaptation (see Section 4.5).

Additional Pareto-frontier visualizations of the feasible quality–cost and quality–latency spaces are provided in the Online Appendix.

Table 4. TopoGuard quality by difficulty level on Water QA. Baseline values are overall averages over the same feasible-context evaluation and should not be interpreted as per-difficulty comparisons.

Method	Easy	Medium	Hard
TopoGuard	**0.920**	**0.915**	**0.851**
Static Workflow (overall)		0.726	
LLM Router (overall)		0.856	

TopoGuard quality by difficulty. Table 4 reports TopoGuard's quality at each difficulty level. TopoGuard's absolute quality decreases from easy ($S = 0.920$) and medium ($S = 0.915$) to hard ($S = 0.851$), consistent with harder instances having noisier profiles and lower base quality. Static Workflow and LLM Router do not adapt topology selection conditioned on difficulty, so their overall scores ($S = 0.726$ and $S = 0.856$, respectively) are reported without per-difficulty breakdowns. The overall gains over Static Workflow and LLM Router are reported in Table 3; Table 4 is intended only to diagnose TopoGuard's own difficulty sensitivity.

Difficulty-sensitivity analysis. TopoGuard remains strong across all difficulty levels, but its absolute score is lower on hard contexts ($S = 0.851$, $n = 51$) than on easy and medium contexts ($n = 85$ each). This pattern indicates that profile-guided selection remains useful under harder instances, while still leaving room for more accurate profile estimation and richer repair actions in the highest-uncertainty cases.

4.3 Per-Context and Pareto Analysis

To further interpret the trade-off, we examine per-context quality differences between TopoGuard and the main reference baselines.

Figure 3 shows the paired comparison between TopoGuard and Static Workflow at the context level. Points above the diagonal correspond to contexts in which TopoGuard achieves higher quality. The win rate is 204/221 (92.3%, 0 ties, 17 losses), with a mean advantage of $\Delta S = +0.176$. Ties require both methods to select identical (topology, executor) pairs with identical realized quality; because TopoGuard's executor assignment always differs from Static Workflow's fixed Kimi-K2.5, zero ties occur. Relative to Best-Quality, the paired comparison shows 0 wins / 136 ties / 85 losses (mean $\Delta S = -0.012$), indicating that TopoGuard trades a small quality loss for substantially lower latency.

4.4 Auxiliary Transfer Evidence: Storm Surge

We also report auxiliary cross-task evidence by applying the same orchestration logic, utility weights, and selection procedure to the Storm Surge benchmark. Domain-specific profiles are estimated from Storm Surge training traces, but the orchestration rule and hyperparameters are not redesigned. To obtain statistically meaningful comparisons, we augment the 30 held-out test tasks with up to six noise-perturbed variants and then apply the same feasibility filtering as in Water QA; therefore, the effective number of valid contexts differs across strategies ($N = 144$–168).

Table 5 shows that TopoGuard reaches $S = 0.886$ in the auxiliary transfer setting, higher than Static Workflow ($S = 0.818$, $\Delta S = +0.068$), and Profile-Cheapest ($S = 0.842$, $\Delta S = +0.047$). Best-Quality achieves $S = 0.896$ but at 24% higher cost ($C = 2.95 \times 10^{-2}$ USD vs 2.38×10^{-2}); TopoGuard stays closer to Best-Quality in quality while using 24% lower cost and 16% lower latency ($S = 0.886$ vs 0.896). An additional point of interest is the topology preference shown in Figure 4: TopoGuard selects a mixture of direct, executor+verifier, and executor+verifier+aggregator prototypes on Water QA (7.7%, 23.1%, and 69.2%, respectively), while shifting further toward executor+verifier+aggregator on Storm Surge (88.1%). Static Workflow remains fixed at executor+verifier

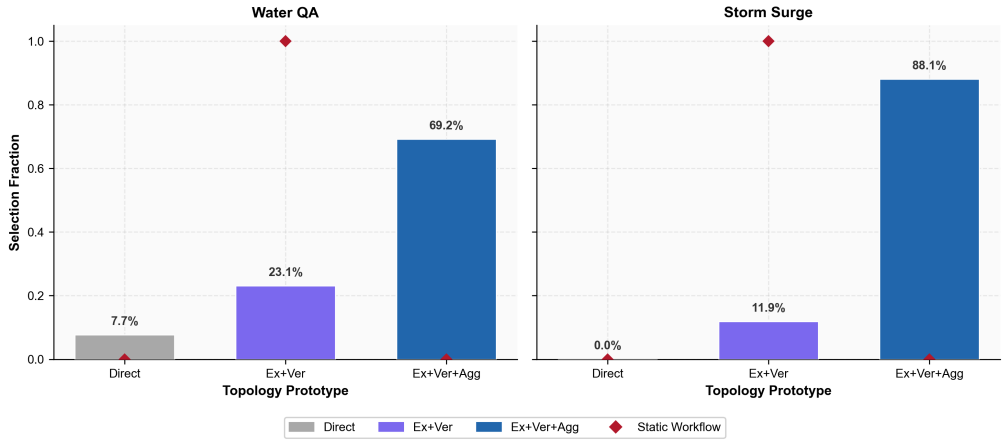

Fig. 4. Topology selection frequency across domains. TopoGuard changes its prototype choices between Water QA and Storm Surge, whereas Static Workflow remains fixed.

Table 5. Auxiliary transfer results on Storm Surge. The noise-perturbed contexts provide supporting evidence rather than full-scale cross-domain validation.

Method	**Quality** $S \uparrow$	**Cost** ($\times 10^{-2}$ **USD**) $\downarrow$	**Latency (s)** $\downarrow$	**N**
TopoGuard	0.886	2.38	9.0	156
Static Workflow	0.818	1.75	40.8	168
Best-Quality	**0.896**	2.95	10.4	150
Profile-Cheapest	0.842	**1.08**	**5.7**	144

in both domains. The selection distributions show that TopoGuard changes its topology choice across domains, whereas Static Workflow keeps the same executor+verifier structure.

Because the Storm Surge result uses noise-perturbed repeats from a smaller held-out set, the transfer result should be read as auxiliary evidence. The consistent topology adaptation and the advantage over the fixed-pipeline baseline are the main takeaways.

4.5 Component Ablation and Repair Analysis

This section addresses **RQ2** and **RQ4**: which components drive the average performance improvement, and whether bounded local repair provides targeted recovery after intermediate execution failures.

Core module ablation (Bootstrap CI + Wilcoxon). Table 6 summarizes the core component analysis on the 221 feasible Water QA contexts. Template and executor ablations are evaluated as paired held-out selection variants. Bounded local repair is part of the full TopoGuard execution pipeline, but its effect is failure-conditioned; therefore, the local-repair row is reported as a diagnostic pre-repair baseline from the frozen-profile recorded-execution replay rather than as a broad average main-effect comparison.

Analysis. Bootstrap 95% CI (1000 resamples, percentile method) gives $S = 0.902$ [0.887, 0.918] for the full system. Template selection and executor adaptation show significant paired gains under the

Table 6. Core module analysis on Water QA. ΔS denotes the quality difference between the full system and each variant. Local repair is an integral execution-time component of TopoGuard, but its measured effect is failure-conditioned; the row is therefore a diagnostic baseline rather than a broad paired main-effect ablation.

Variant	**Quality** $S \uparrow$	**N**	ΔS **(full − ablated)** $\uparrow$
TopoGuard (full)	**0.902**	221	—
w/o Template Selection	0.844	221	+0.059
w/o Executor Adaptation	0.793	221	+0.109
w/o Local Repair	0.893	221	+0.009

Table 7. Failure-conditioned local repair statistics on Water QA.

Statistic	**Value**
Diagnostic pre/post repair gap	+0.009 over feasible recorded-execution replay co
Triggered bounded repair	231 / 585 monitored contexts
Repair-triggered low-quality executions left unchanged w/o repair	231 / 585 monitored contexts
Mean gain per triggered repair	+0.0277
Dominant non-null routes	Executor upgrade: 97.0%, C2: 2.2%, C1: 0.9%

Wilcoxon signed-rank test. In the paired main-strategy table, the TopoGuard and w/o-Local-Repair records are tied because most contexts do not change the selected final candidate after repair gating; therefore, repair should be read through the failure-conditioned diagnostic below rather than as a broad average-performance driver.

Contribution ranking shows that executor adaptation and topology-prototype selection are the main average performance drivers. Removing **Executor Adaptation** has the largest impact ($\Delta S = +0.109$), while removing **Template Selection** yields $\Delta S = +0.059$. The local-repair diagnostic shows a smaller pre/post gap ($\Delta S = +0.009$) and is not statistically significant as a paired main effect. This does not make repair optional: it indicates that repair is not the primary source of TopoGuard's average quality gain under nominal recorded-execution replay, while remaining the execution-time recovery layer activated after detected local failures.

Repair mechanism analysis (RQ4). In the closed-loop diagnostic, 231 of 585 monitored Water QA execution contexts trigger bounded repair (39.5%), all activated by realized intermediate quality signals. In the corresponding no-repair condition, these triggered executions would be left unchanged after falling below the repair pass threshold. The mean gain over triggered repairs is +0.0277, larger than the average gain over all feasible contexts. Non-null repair actions are dominated by executor upgrades within the current topology (224 cases, 97.0%), with smaller contributions from evaluator/verifier adjustment (5 cases, 2.2%) and cross-topology executor repair (2 cases, 0.9%).

Complex-task local-failure stress diagnostic. Because the nominal recorded-execution replay contains many contexts whose initially selected workflow already exceeds the pass threshold, we additionally evaluate repair under a small set of predeclared complex execution-time failures. The profiles and initial selection policy are kept frozen; only held-out test execution is perturbed, and the diagnostic is not used to tune the profiles or the main strategy comparison. In these controlled local-failure diagnostics, bounded repair recovers more quality for local node, reasoning–computation, and verifier failures than for global evidence degradation; detailed stress results are reported in the Online Appendix.

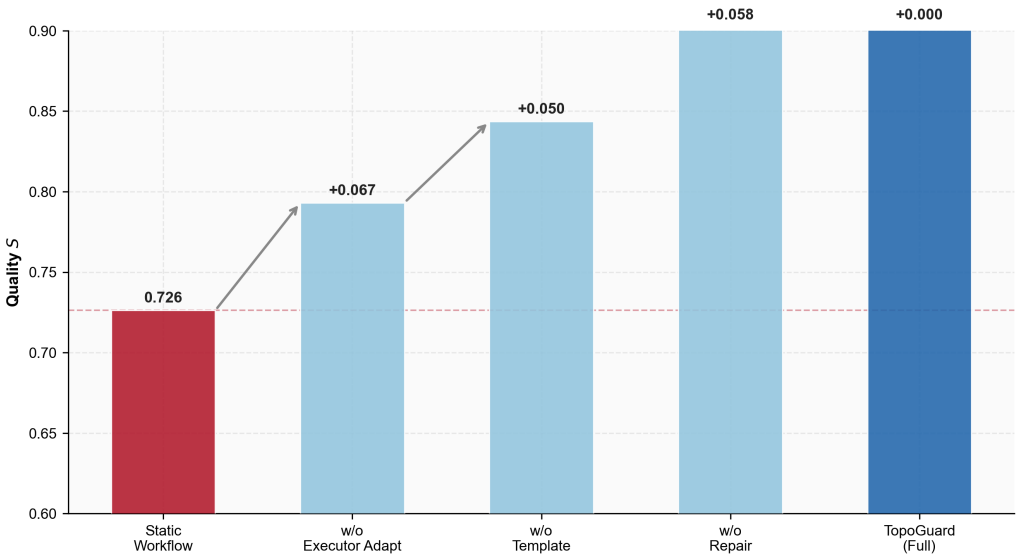

Fig. 5. Stepwise component ablation. Executor adaptation and topology selection account for most of the average gain; local repair acts as a failure-conditioned safeguard.

Simplified repair evaluation. The replay-based repair diagnostic ranks repair candidates by frozen profile estimates before observing realized test quality, but it does not fully model all online switching overheads. Online repair execution would need to apply edit-loss to balance repair benefit against switching overhead; we leave this as future work.

Figure 5 provides a visual summary of the stepwise contribution of executor adaptation, topology selection, and local repair.

4.6 Sensitivity to Utility Weights and Constraint Robustness

We next consider **RQ3** from the perspective of utility weights and hard constraints. A useful orchestration policy should react to changes in utility weights in a predictable way, without becoming unstable under moderate reweighting or moderate profile uncertainty [2, 16].

To test this, we vary (α, β, γ) and re-score candidates on the same held-out contexts without retraining the profiles. Seven representative settings are considered, including the default configuration, quality-dominant weighting, balanced weighting, cost-priority weighting, latency-priority weighting, and two partial-objective variants. The selected workflows shift in an interpretable way as the weights change: quality-dominant weighting reaches $S = 0.912$, while latency-priority weighting reduces latency at the expected cost of quality ($S = 0.705$). Across all tested settings, quality remains within $S \in [0.705, 0.914]$, suggesting that the method is sensitive to utility preferences but not excessively fragile. Detailed weight-sensitivity results are reported in the Online Appendix.

Constraint-related robustness. Because the main experiment uses hard filtering before selection, all reported strategies satisfy the estimated cost and latency constraints by construction. We therefore do not treat violation rate as a discriminative metric in the main experiment. Instead, we report feasible coverage under tightened global cost budgets and discuss profile drift as future work.

Constraint stress test. We also evaluate how TopoGuard behaves when the normalized cost budget is varied. In this diagnostic, the same 221 of 255 contexts remain feasible across representative budget levels, and the selected average operating point remains unchanged ($S = 0.902$, $C = 6.402 \times 10^{-3}$ USD). This should be read as a sanity check rather than strong robustness evidence: under the present normalized budget scale, infeasibility is driven mainly by contexts with no viable candidate in the template library rather than by small changes around the default budget. Detailed constraint and modality-perturbation diagnostics are reported in the Online Appendix.

The aggregate results characterize the overall trade-off, but they do not show how the framework behaves within a single run. To make the execution behavior more concrete, we include a case study based on a storm-surge warning task.

4.7 Case Study: Execution and Local Repair in a Storm-Surge Scenario

We conduct the case study on a water-conservancy digital-twin agent platform. The platform integrates three key components: multimodal monitoring data, a task decomposition interface, and a pool of executable domain agents. This environment provides a realistic setting for evaluating how TopoGuard organizes available executors into an executable workflow for storm-surge warning tasks.

For a storm-surge warning query, the selected workflow combines textual retrieval, time-series and spatial computation, scalar verification, multimodal reasoning, and final aggregation. In the concrete execution, the default computation node underestimates uncertainty in the typhoon track forecast, producing verification scores below the pass threshold. Instead of replanning the entire workflow, TopoGuard triggers bounded local repair and replaces the computation node with a more robust variant that accounts for forecast spread while preserving reusable upstream outputs. Detailed workflow artifacts and the corresponding multimodal evidence table are provided in the Online Appendix.

The case illustrates the role of bounded local repair in high-stakes settings: the aggregate contribution of repair is small because most contexts either do not trigger repair or select the same final candidate, but the triggered-repair diagnostic shows a positive mean gain (+0.0277). In risk-sensitive domains, this targeted safeguard can reduce downstream decision failures that could propagate into incorrect warnings or missed alerts.

4.8 Limitations and Future Work

Scope of multimodal modeling. The present study addresses multimedia decision support at the workflow-structure level: different nodes consume textual, temporal, spatial, scalar, and probabilistic artifacts, and TopoGuard selects verification- or aggregation-enhanced topologies when the required modality combination or evidence reliability changes. It does not study new end-to-end fusion architectures or representation-completion models; a full missing-modality benchmark with controlled corruption of each modality is left as future work.

Representativeness of experimental scenarios. Both Water QA and Storm Surge come from the same water-conservancy digital-twin platform with relatively limited sample sizes. The Storm Surge auxiliary transfer experiment covers fewer (node type, difficulty) combinations than Water QA, constituting small-sample supporting evidence rather than full-scale validation. Generalization claims across domains require verification in more diverse real-world settings.

Hard-constraint stress testing. The main experiment enforces ex-ante estimated feasibility by pre-filtering candidates against budget and latency constraints. This design isolates the effect of selection heuristics under the same feasible candidate space, but it does not fully characterize behavior when the feasible set becomes sparse or empty. The constraint stress test in Table 12 partially addresses this issue by tightening the cost budget and reporting feasible coverage; more

extensive stress tests with larger template libraries, cold-start profiles, and explicit empty-set recovery policies remain future work.

External validity of the recorded-execution replay protocol. All main results are based on a frozen-profile recorded-execution replay protocol using historical profile lookups, not a live online deployment. This protocol does not account for executor performance drift over time or cold-start scenarios where new tasks lack historical profiles. A more realistic dynamic profile update mechanism (e.g., online Bayesian updates in the profile manager) has not been validated end-to-end.

Representativeness of the static baseline. The Static Workflow configuration (executor+verifier + Kimi-K2.5 executor) is a representative fixed-topology choice under the experimental budget constraints, not a hand-optimized strongest static topology. The equal-budget fixed-topology analysis provides an initial check against stronger fixed alternatives, but a broader suite of hand-designed fixed workflows and domain-specific pipelines would further strengthen external validity.

Proxy comparison with workflow generation methods. We include an AFlow-Style Offline Search baseline instantiated within the same auditable topology space, but this is not a full reproduction of AFlow, AutoFlow, or WorkflowLLM. Full workflow-generation systems target unconstrained offline structure optimization on different benchmarks such as code generation and planning. Directly adapting them to our hard cost–latency constraints and multimodal decision setting remains future work.

Empirically set edit-loss weights. The edit-loss weights $\lambda_n = 0.30$, $\lambda_\phi = 0.25$, etc. are set from engineering experience and have not been systematically optimized. Sensitivity analysis for τ_{pass} and systematic experiments for the edit-loss weights remain as future work.

These limitations delimit the scope of the present results and point to important directions for future validation.

4.9 Discussion

TopoGuard improves quality relative to a fixed static workflow ($S = 0.902$ vs. 0.726, $\Delta S = +0.176$), while staying close to the quality-maximizing baseline on Water QA (0.902 vs. 0.914) and within 1.1 percentage points of Best-Quality on Storm Surge. The experiments show that TopoGuard mainly improves the quality side of the feasible quality–cost–latency trade-off, while avoiding the high-latency Best-Quality operating point.

The cost-adjusted paired regression shows a positive intercept ($\beta_0 = +0.1266$), suggesting that TopoGuard's quality advantage includes a component not explained by cost increase alone. The ablation results more directly show that executor adaptation and topology-prototype selection account for most of the average gain. TopoGuard achieves $S = 0.920$ on easy, $S = 0.915$ on medium, and $S = 0.851$ on hard contexts, with an overall advantage of $\Delta S = +0.176$ over the static baseline ($S = 0.726$). Bounded local repair is the execution-time adaptation layer of TopoGuard. It is not a significant paired main-effect driver under nominal recorded-execution replay, but it provides a mean gain of +0.0277 when triggered and larger recovery in a small controlled local-failure diagnostic. Executor upgrade within the current topology is the dominant repair route. Bounded local repair is therefore best interpreted as a targeted execution-time safeguard rather than the primary average performance driver.

5 Conclusion

This paper presented TopoGuard, a profile-guided adaptive topology orchestration framework for risk-sensitive multimodal decision support. TopoGuard maintains quality–cost–latency profiles for candidate auditable workflow DAG prototypes and executor assignments, selects an initial feasible topology through constrained Pareto pruning, and applies bounded local repair when intermediate

execution quality degrades. A trace-based evaluation with frozen profiles over execution records from a water-conservancy digital-twin platform shows that TopoGuard stays close to the quality-maximizing baseline on Water QA ($S = 0.902$ vs. 0.914) while reducing latency (77.6 s vs. 118.9 s), and improves over fixed workflows, executor-only routing, and offline-search proxy baselines in the feasible replay comparison. The component, difficulty-wise, and repair analyses suggest that the full TopoGuard system improves over the static baseline by $\Delta S = +0.176$. Executor adaptation and topology-prototype selection are the main average performance drivers, while bounded local repair serves as the failure-conditioned execution-time adaptation layer: it is not significant as a paired main effect under nominal recorded-execution replay, but it yields a mean gain when triggered (+0.0277) and larger recovery in a small controlled local-failure diagnostic.

References

[1] Nilani Algiriyage, Raj Prasanna, Kristin Stock, Emma E. H. Doyle, and David Johnston. 2022. Multi-source Multimodal Data and Deep Learning for Disaster Response: A Systematic Review. *SN Computer Science* 3, 1 (2022), 92. doi:10.1007/s42979-021-00971-4

[2] Tian Cao, Fang Liu, Yu Zhang, and Yi Chen. 2024. Online Learning for Adaptive Risk-Sensitive Systems. In *Proceedings of the International Conference on Machine Learning*. PMLR, 11234–11247.

[3] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research* (2024). arXiv:2305.05176 [cs.CL]

[4] Dickson K. W. Chiu, Qing Li, and Kamalakar Karlapalem. 2001. Web Interface-Driven Cooperative Exception Handling in ADOME Workflow Management System. *Information Systems* 26, 2 (2001), 93–120. doi:10.1016/S0306-4379(01)00012-6

[5] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. 2024. A Unified Approach to Routing and Cascading for LLMs. *arXiv preprint arXiv:2410.10347* (2024). arXiv:2410.10347 [cs.CL] doi:10.48550/arXiv.2410.10347

[6] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. *arXiv preprint arXiv:2411.05451* (2024). arXiv:2411.05451 [cs.CL] doi:10.48550/arXiv.2411.05451

[7] Xiang Gao, Qiang Liu, Siyu Chen, Lin Zou, and Fan Wang. 2024. Constraint-Aware Routing for Cloud-Based AI Services. *IEEE Transactions on Network and Service Management* 21, 3 (2024), 1198–1211. doi:10.1109/TNSM.2024.3390120

[8] Chenyu Ge and Shengfeng Qin. 2025. Urban Flooding Digital Twin System Framework. *Systems Science and Control Engineering* 13, 1 (2025), 2460432. doi:10.1080/21642583.2025.2460432

[9] Luis F. Gonzalez, Juan Tan, and Balasubramaniam Srinivasan. 2022. Constrained Multi-Objective Optimization for Hybrid Decision Systems under Uncertainty. *Journal of Optimization Theory and Applications* 174 (2022), 432–455. doi:10.1007/s10957-021-03005-8

[10] Bolin Huang, Zhong Wu, Qiang Li, and Feng Zhang. 2023. DAG Scheduling with Deadlines and Resource Constraints in Uncertain Environments. In *International Conference on Automated Planning and Scheduling*. 178–187. doi:10.1609/icaps.v33i1.21818

[11] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the Planning of LLM Agents: A Survey. *arXiv preprint arXiv:2402.02716* (2024). arXiv:2402.02716 [cs.AI] doi:10.48550/arXiv.2402.02716

[12] Dominik Hutter, Steffen Wolf, Berk Hermann, and Matteo Pinciroli. 2023. Online Failure Recovery for Workflow Execution with Adaptive Checkpointing. In *Proceedings of the International Conference on Software Engineering*. 1187–1199. doi:10.1145/3510003.3510158

[13] Solomon Inyang and Firouzeh Rosa Taghikhah. 2025. Digital Twin-Based Decision Support Systems for Natural Disaster Management: A Systematic Review of Current Trends and Approaches. *Smart Cities and Transport* (2025), 100406. doi:10.1016/j.sctalk.2024.100406

[14] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 24370–24391.

[15] Yuna Kim, Wan Yeon Lee, Kyong Hoon Kim, and Jong Kim. 2010. AdaFF: Adaptive Failure-Handling Framework for Composite Web Services. *IEICE Transactions on Information and Systems* E93-D, 4 (2010), 834–842. doi:10.1587/transinf.E93.D.834

[16] Paul N. Koch, Barry Wujek, and Oleg Golovidov. 2019. Robust Optimization and Risk-Sensitive Decision Making in Complex Engineering Systems. *Structural and Multidisciplinary Optimization* 59 (2019), 2061–2083. doi:10.1007/s00158-018-2104-0

[17] Guilin Lan, Ye-Qian Du, and Zhouwang Yang. 2025. Robust Multimodal Representation Learning for Missing Modalities. *ACM Transactions on Multimedia Computing, Communications, and Applications* 21, 1 (2025), 34:1–34:23. doi:10.1145/3702003

[18] Wengang Li, Zifei Ma, Jing Li, Qinghua Li, Yang Li, and Juan Yang. 2024. Digital Twin Smart Water Conservancy: Status, Challenges, and Prospects. *Water* 16, 14 (2024), 2038. doi:10.3390/w16142038

[19] Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. AutoFlow: Automated Workflow Generation for Large Language Model Agents. *arXiv preprint arXiv:2407.12821* (2024). arXiv:2407.12821 [cs.AI] doi:10.48550/arXiv.2407.12821

[20] Muhammad Irzam Liaqat, Shah Nawaz, Muhammad Zaigham Zaheer, Muhammad Saeed, Hamza Sajjad, Toon De Schepper, Karthik Nandakumar, Muhammad Haris Khan, Ignazio Gallo, and Markus Schedl. 2025. Chameleon: A Multimodal Learning Framework Robust to Missing Modalities. *International Journal of Multimedia Information Retrieval* 14, 21 (2025). doi:10.1007/s13735-025-00370-y

[21] Chen Liu, Jia Wu, Yefan Zheng, Tong Luo, and Wei Zhao. 2024. Risk-Aware Task Offloading for Edge-Cloud Collaborative Multimedia Systems. *IEEE Transactions on Mobile Computing* 23, 7 (2024), 7892–7907. doi:10.1109/TMC.2023.3345821

[22] Kaisa Miettinen, Francisco Ruiz, and Margaret M. Wiecek. 2010. Interactive Nonlinear Multiobjective Optimization Procedures. *Handbook on Multicriteria Decision Analysis* (2010), 153–196. doi:10.1007/978-0-387-74713-0_6

[23] Drashti Pathak, Aparna S. Varde, Gerard de Melo, and Clement Alo. 2020. Hydroinformatics and the Web: Analytics and Dissemination of Hydrology Data for Climate Change and Sustainability. *ACM SIGWEB Newsletter* Autumn (2020). doi:10.1145/3427478.3427481

[24] Uri Peled, Arin Jacob, and Assaf Goldberger. 2021. Graph Neural Networks for Graph-Level Tasks: A Survey. *arXiv preprint arXiv:2109.10832* (2021). arXiv:2109.10832 doi:10.48550/arXiv.2109.10832

[25] Yuxin Qin et al. 2023. A Review of Application of Machine Learning in Storm Surge Problems. *Journal of Marine Science and Engineering* 11, 9 (2023), 1729. doi:10.3390/jmse11091729

[26] Md. F. Reza et al. 2024. Robust Multimodal Learning with Missing Modalities via Parameter-Efficient Adaptation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024). doi:10.1109/TPAMI.2024.3476487

[27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. arXiv:2302.04761 [cs.CL]

[28] Yixuan Sun, Xin Fan, Ziqi Wang, Lingxi Liu, and Zheng Zhou. 2023. Pareto-Optimal Sensor Selection for Multimodal Decision Systems under Budget Constraints. *IEEE Sensors Journal* 23, 11 (2023), 11234–11247. doi:10.1109/JSEN.2023.3268402

[29] John C. Tang, Siripen Pongpaichet, and Ramesh Jain. 2016. Research Challenges in Developing Multimedia Systems for Managing Emergency Situations. *ACM Transactions on Multimedia Computing, Communications, and Applications* 12, 4s (2016), 1–18. doi:10.1145/2964284.2976761

[30] Xin Tian, Jing Wang, Yang Liu, and Wei Chen. 2024. Constraint-Aware Sequence-to-Sequence Learning for Resource-Limited Decision Systems. *Artificial Intelligence Review* 57 (2024), 1–24. doi:10.1007/s10462-023-10656-2

[31] Arkadiusz Tomczak, Grzegorz Stepien, Tomasz Kogut, Lukasz Jedynak, Grzegorz Zaniewicz, Malgorzata Lacka, and Izabela Bodus-Olkowska. 2025. Development of a Digital Twin of the Harbour Waters and Surrounding Infrastructure Based on Spatial Data Acquired with Multimodal and Multi-Sensor Mapping Systems. 315 pages. doi:10.3390/app15010315

[32] Yuhan Wang, Xing Zhang, Sijia Liu, Yang Chen, and Dong Xu. 2023. Adaptive Workflow Recovery for Multi-Stage Decision Pipelines. In *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases*. 445–461. doi:10.1007/978-3-031-43427-3_41

[33] Renjie Wu, Hu Wang, Hsiang-Ting Chen, and Gustavo Carneiro. 2024. Deep Multimodal Learning with Missing Modality: A Survey. *arXiv preprint arXiv:2409.07825* (2024). arXiv:2409.07825 [cs.CV] doi:10.48550/arXiv.2409.07825

[34] Ke Yang, Jiateng Liu, John Wu, Chaoqi Yang, Yi R. Fung, Sha Li, Zixuan Huang, Xu Cao, Xingyao Wang, Yiquan Wang, Heng Ji, and ChengXiang Zhai. 2024. If LLM Is the Wizard, Then Code Is the Wand: A Survey on How Code Empowers Large Language Models to Serve as Intelligent Agents. *arXiv preprint arXiv:2401.00812* (2024). arXiv:2401.00812 [cs.CL] doi:10.48550/arXiv.2401.00812

[35] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. arXiv:2305.10601 [cs.CL]

[36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

[37] Xiaoya Yu, Kejun Wu, You Yang, and Qiong Liu. 2024. WaRENet: A Novel Urban Waterlogging Risk Evaluation Network. *ACM Transactions on Multimedia Computing, Communications, and Applications* 20, 7 (2024), 1–28. doi:10.1145/3651163

[38] Min Yuan, Siyuan Wang, Huan Chen, and Qing Liu. 2024. Digital Twin for Industrial Control Systems: A Systematic Review of Architecture and Applications. *IEEE Transactions on Industrial Informatics* 20, 6 (2024), 8283–8294. doi:10.1109/TII.2024.3371023

[39] Aohan Zeng, Mingdao Liu, Ruxiao Du, et al. 2023. FlowMind: Automatic Workflow Generation with LLMs. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 73–81. doi:10.1145/3604237.3626908

[40] Chen Zhang, Yikang Liu, Ruyu Xu, Hui Wang, and Xiaolan Fu. 2023. Multimodal Fusion for Risk Assessment in Critical Infrastructure Systems. *ACM Transactions on Multimedia Computing, Communications, and Applications* 19, 2 (2023), 52:1–52:23. doi:10.1145/3564992

[41] Duzhen Zhang, Yahan Yu, Jiahua Dong, Chenxing Li, Dan Su, Chenhui Chu, and Dong Yu. 2024. MM-LLMs: Recent Advances in Multimodal Large Language Models. *arXiv preprint arXiv:2401.13601* (2024). arXiv:2401.13601 [cs.CV] doi:10.48550/arXiv.2401.13601

[42] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2024. AFlow: Automating Agentic Workflow Generation. *arXiv preprint arXiv:2410.10762* (2024). arXiv:2410.10762 [cs.AI] doi:10.48550/arXiv.2410.10762

[43] Wei Zhou, Ming Chen, Dong Li, Li Shen, and Xin Wang. 2023. Foundation Model for End-to-End Multimodal Reasoning in Risk-Sensitive Decision Support. *IEEE Transactions on Knowledge and Data Engineering* (2023). doi:10.1109/TKDE.2023.3314122

Online Appendix

A. Additional Pareto-Frontier Visualization

Figure 7 shows the feasible quality–cost and quality–latency spaces for the Water QA candidate pool. This visualization is auxiliary to the main Water QA comparison in Table 3; it illustrates the trade-off geometry but does not define an additional evaluation metric.

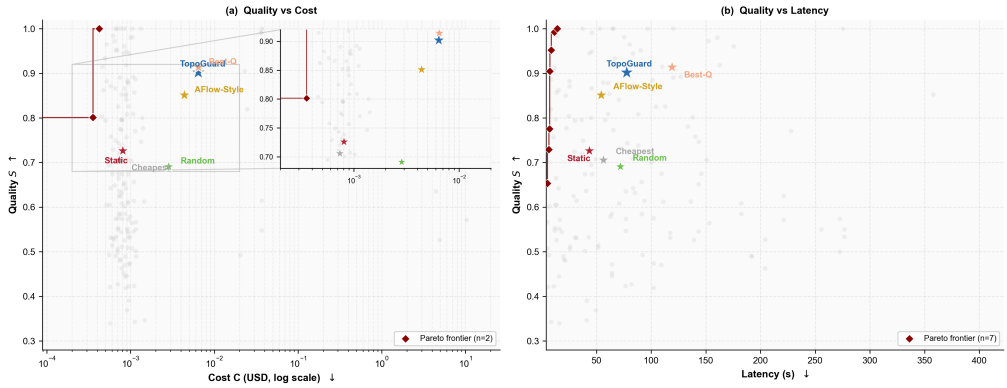

Fig. 6. Auxiliary Pareto-frontier visualization of the feasible Water QA candidate space. Left: quality–cost trade-off. Right: quality–latency trade-off. Strategy markers show average operating points.

B. Complex Local-Failure Stress Diagnostic

For the complex-task repair diagnostic, profiles and the initial selection policy are kept frozen; only held-out test execution is perturbed. Local-failure scenarios degrade the selected node output as a controlled proxy for an executor or intermediate multimodal artifact failing on a complex instance, while repair candidates are selected from the same profile library before their realized quality is observed. Table 10 reports the detailed stress results.

Table 8. Complex-task local-failure stress diagnostic for TopoGuard. n denotes predeclared stress context groups; S_{shock} is the quality after test-time degradation before repair.

Stress scenario	n	S_{nominal}	S_{shock}	S_{repair}	ΔS
Hard local node failure	3	0.854	0.495	0.606	+0.110
Hard reasoning–computation failure	3	0.854	0.470	0.732	+0.263
Medium–hard verifier failure	3	0.787	0.456	0.622	+0.166
Hard global evidence degradation	3	0.854	0.641	0.661	+0.021

The diagnostic clarifies the operational boundary of repair. When the failure is local to a node or executor, bounded repair recovers part of the lost quality without global replanning. When the degradation is global to the evidence stream, the improvement is much smaller because replacement candidates face the same degraded evidence.

C. Utility, Constraint, and Perturbation Diagnostics

Table 11 reports the utility-weight sensitivity analysis. The selected workflows shift in an interpretable way as the weights change, and the quality range remains bounded across the tested settings.

Table 9. Sensitivity analysis under different utility weight settings.

Weight Setting	α	β	γ	$S \uparrow$
Default (quality-centric)	0.65	0.25	0.10	0.902
Quality-dominant	0.92	0.05	0.03	0.912
Balanced	0.50	0.30	0.20	0.824
Cost-priority	0.40	0.50	0.10	0.853
Latency-priority	0.10	0.10	0.80	0.705
Q+C only	0.80	0.20	0.00	**0.914**
Q+L only	0.80	0.00	0.20	0.853

Table 10. Constraint stress test under tightened cost budgets. Quality is computed only over remaining feasible contexts.

Budget Level	**Feasible%**	$S \uparrow$	C ($\times 10^{-3}$ USD) $\downarrow$	**Topology Shift**
$C_{max} = 0.30$	86.7%	0.902	6.402	stable
$C_{max} = 0.50$	86.7%	0.902	6.402	stable
$C_{max} = 0.70$	86.7%	0.902	6.402	stable

As a controlled evidence-degradation diagnostic, we reduce quality estimates for specific modality-consuming node types. Removing spatial, textual, or scalar evidence causes 17 topology shifts in the affected settings, while the noisy-sensor perturbation leaves the selected topology distribution unchanged. This diagnostic suggests that topology selection can respond to modality-specific profile changes, but more realistic perturbations where relative rankings within a node type change are needed to fully evaluate multimodal uncertainty adaptation.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.